ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ НЕФТИ И ГАЗА
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
ИМЕНИ И.М. ГУБКИНА»

ФАКУЛЬТЕТ КОМПЛЕКСНОЙ БЕЗОПАСНОСТИ ТЭК
КАФЕДРА БЕЗОПАСНОСТИ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

Лабораторная работа №9

по дисциплине «АДМИНИСТРИРОВАНИЕ ОПЕРАЦИОННЫХ СИСТЕМ»

на тему «Работа с процессами и заданиями»

Выполнила студентка:
группы КА-22-06
Новикова Дарья Дмитриевна

Преподаватель:
Уймин Антон Григорьевич

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 3

ОСНОВНАЯ ЧАСТЬ 4

КОНТРОЛЬНЫЕ ЗАДАНИЯ 23

ЗАКЛЮЧЕНИЕ..... 33

КОНТРОЛЬНЫЕ ВОПРОСЫ 34

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ.. 36

ВВЕДЕНИЕ

Цель работы:

- Ознакомиться с управлением процессами и заданиями в операционной системе ALT Linux, научиться взаимодействовать с процессами с помощью командной строки, отслеживать состояние процессов и управлять заданиями. Понять основы многозадачности и приоритетов процессов в UNIX-подобных системах.

Оборудование, ПО:

- Машина CLI с подключенными тремя внешними жесткими дисками
- Справочная литература или доступ в сеть интернет.

ОСНОВНАЯ ЧАСТЬ

Выполняющаяся что-либо в системе программа называется процессом. Каждый процесс имеет свое собственное пространство памяти, и может быть запущен в определенном состоянии (например, готов к выполнению, в ожидании, заблокирован и т. д.). Таким образом, процесс включает в себя информацию о состоянии выполнения программы (например, инструкции, регистры процессора, стек вызовов и переменные) и ресурсы, необходимые для работы (например, память, файлы, устройства ввода-вывода). Операционная система управляет процессами, назначая им ресурсы, устанавливая приоритеты выполнения, планируя их работу и обеспечивая безопасность выполнения.

Все процессы система регистрирует в таблице процессов, присваивая каждому уникальный номер – идентификатор процесса (process identificator, PID).

Чтобы получить список всех процессов в расширенном формате, необходимо в терминале ввести команду (рис. 1):

$ ps -ef

-e — показать все процессы.

-f — полный формат вывода (включает UID, PPID, C, STIME, TTY, TIME, CMD).

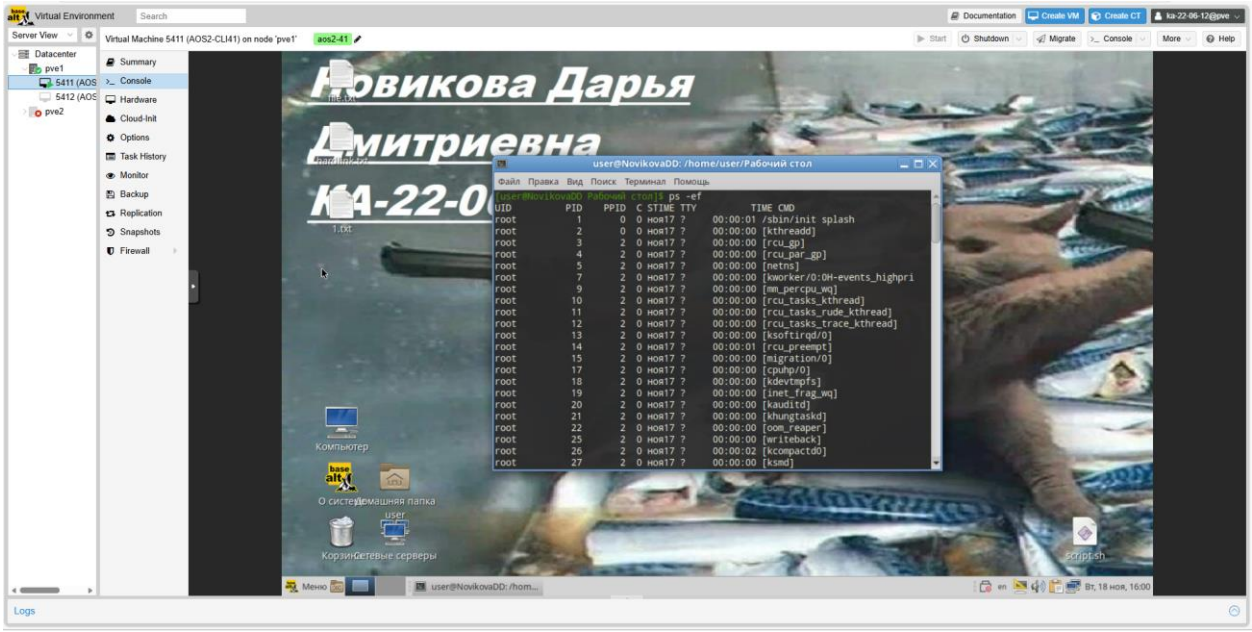

Рисунок 1 — Все процессы в деталях

Каждая строка вывода представляет один процесс и содержит информацию о его идентификаторе (PID), пользователе, использовании CPU, памяти и других характеристиках.

Расшифровка столбцов ps -ef:

- UID — пользователь-владелец процесса
- PID — уникальный ID процесса для управления
- PPID — ID родительского процесса (иерархия)
- C — загрузка CPU процессом (%)
- STIME — время запуска процесса
- TTY — терминал:
- • ? — демон (без терминала)
- • pts/X — SSH/терминал
- • ttyX — консоль
- TIME — общее время использования CPU
- CMD — команда запуска процесса с аргументами

Состояния процессов

R — выполняется или готов к выполнению (в очереди к CPU)

S — спит (ожидает ресурсы)

D — беспробудный сон (непрерываемый ввод-вывод)

T — остановлен сигналом

Z — зомби (завершен, но не удален из таблицы процессов)

I — бездействующий поток ядра

Дополнительные флаги: < — высокий приоритет, N — низкий приоритет, + — foreground группа

Приоритетные/фоновые процессы

Приоритетные — привязаны к терминалу, блокируют ввод до завершения

Фоновые — работают независимо от терминала, не блокируют ввод

Запуск в фоне: команда &

Процессы можно переключать между foreground/background режимами

Приоритетные задачи можно также превращать в фоновые и возвращать обратно в приоритетные.

Пример (рис. 2):

$ ping ya.ru

Нажатие сочетания клавиш Ctrl-Z приведет к остановке данного процесса.

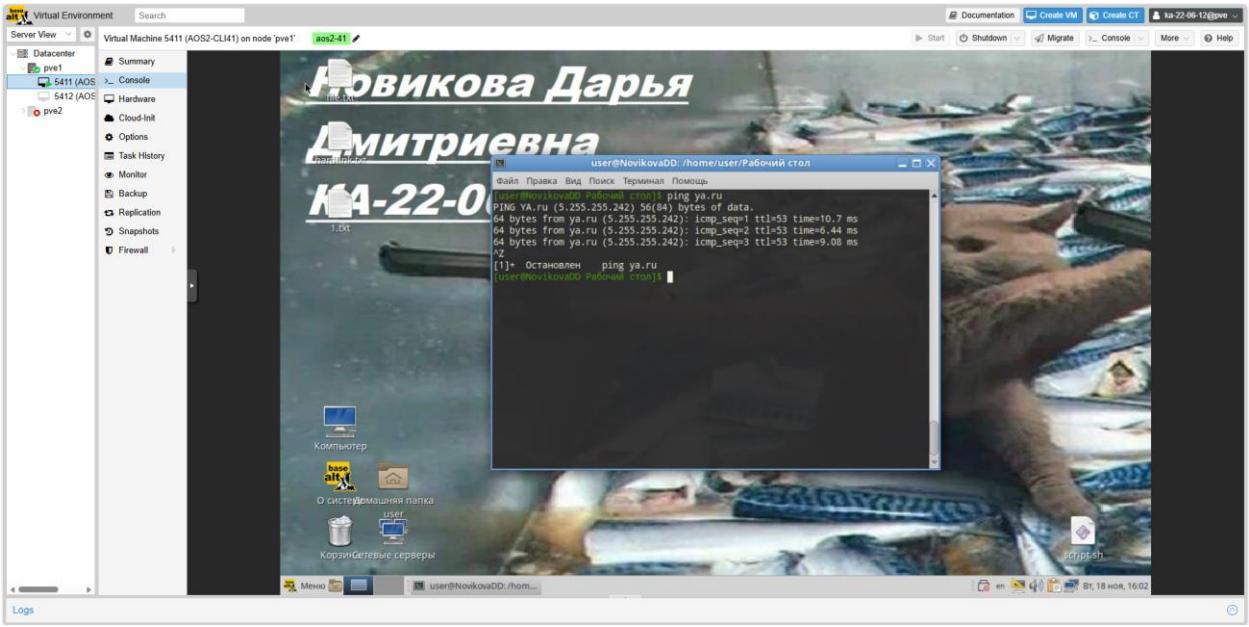

Рисунок 2 — Результат работы ping ya.ru

Далее остановленный процесс можно снова запустить в фоновом режиме с помощью команды *bg [команда]*.

Результат работы (рис. 3):

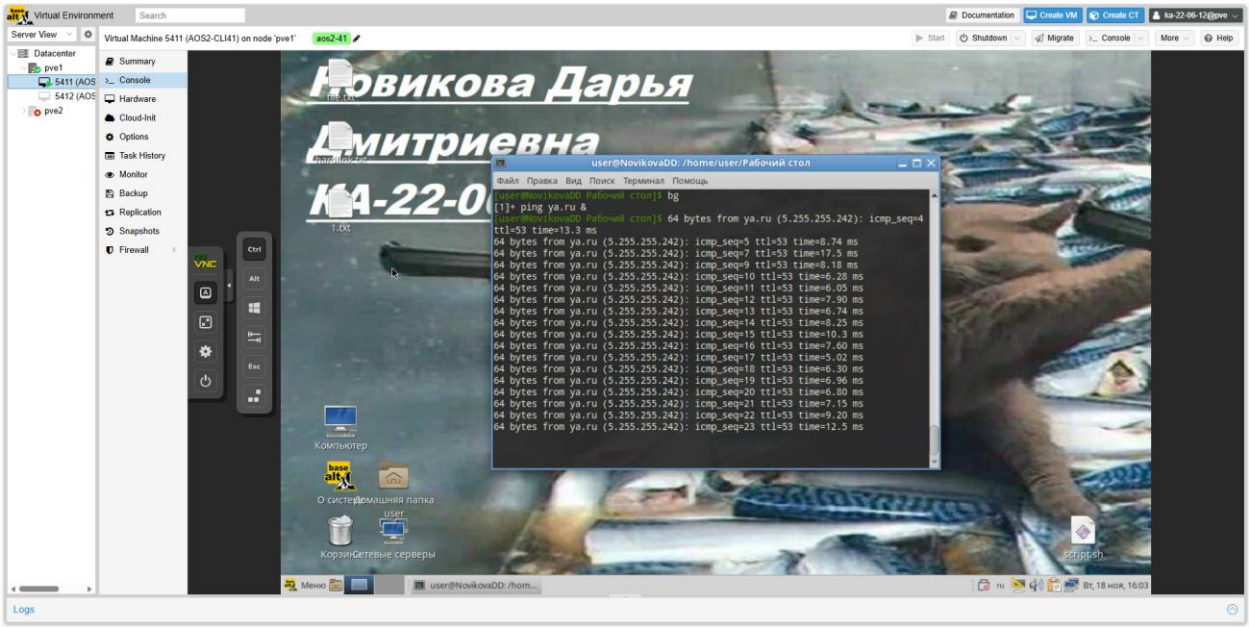

Рисунок 3 — Запуск в фоне

Чтобы вернуть фоновый или приостановленный процесс на передний план воспользуемся командой *fg [команда]* (рис. 4).

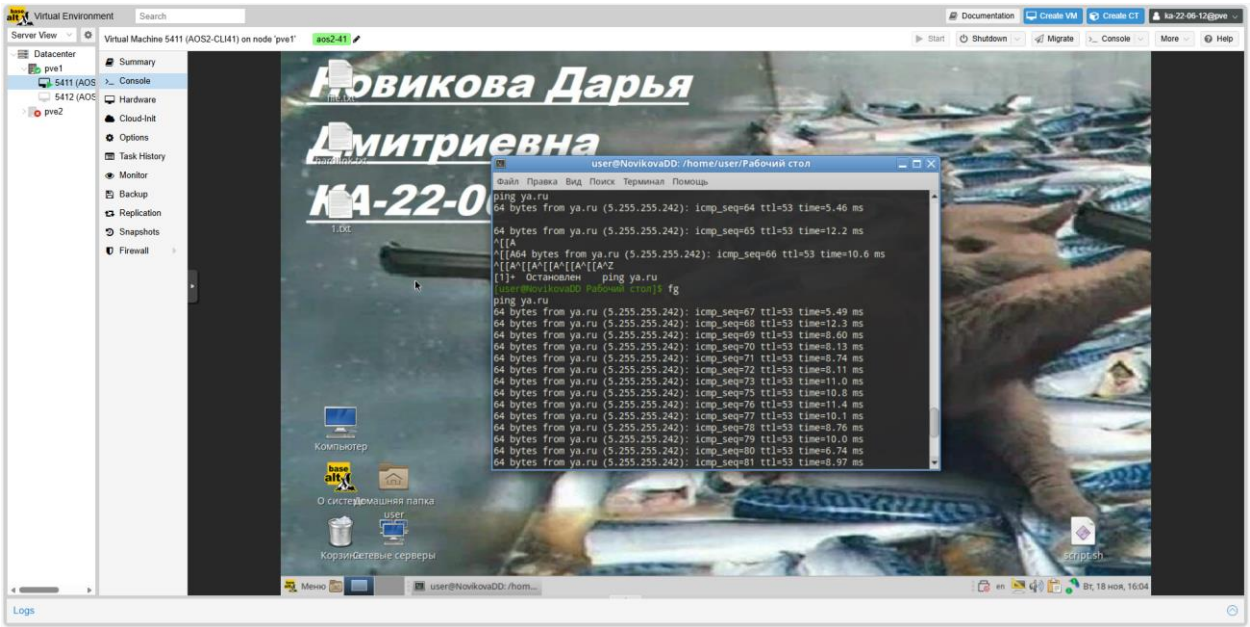

Рисунок 4 — Возврат на передний план

Команды fg и bg переводят из/в фоновый режим задание с указанным номером. Если номер не указан, действие применяется к последнему созданному заданию.

Иерархия процессов в системе

В ОС Альт роль начального процесса выполняет systemd, который заменяет традиционный init. Systemd является системным и сервисным менеджером; родительским процессом для всех остальных процессов; отвечает за монтирование ФС, управление устройствами и запуск системных служб

Утилита pstree отображает дерево запущенных процессов, показывая их иерархическую связь (рис. 5). При вызове утилиты pstree без каких-либо параметров, она отобразит дерево процессов всей системы. Верхний / корневой элемент в дереве является родительским процессом для всех системных процессов.

$ pstree

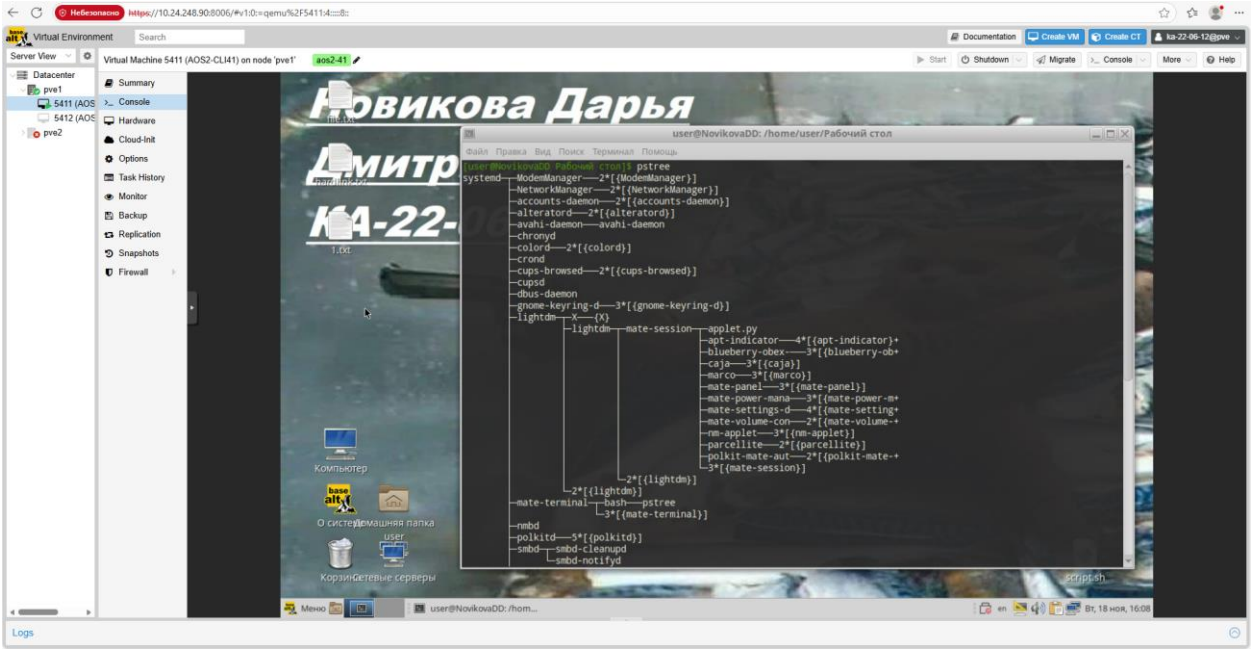

Рисунок 5 — Дерево процессов системы

Утилита объединяет идентичные ветви, помещая их в квадратные скобки и ставя перед ними целое число, представляющее количество ветвей. Это делает вывод более читаемым и визуально привлекательным.

Чтобы отключить слияние идентичных веток, воспользуемся опцией –c (рис. 6):

$ pstree –c

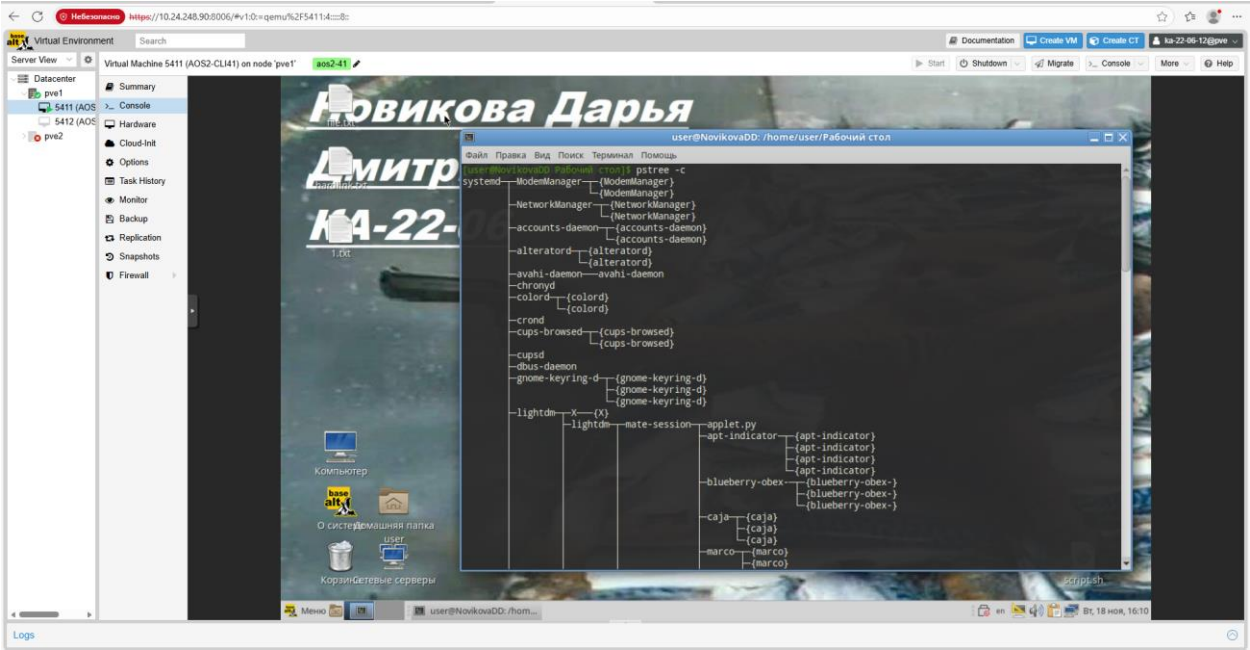

Рисунок 6 — Дерево без сворачивания

Потоки процесса показаны под родительским процессом и отображаются с использованием имени процесса в фигурных скобках.

Чтобы показать полные имена потоков, воспользуемся опцией -t (рис. 7):

$ pstree -t

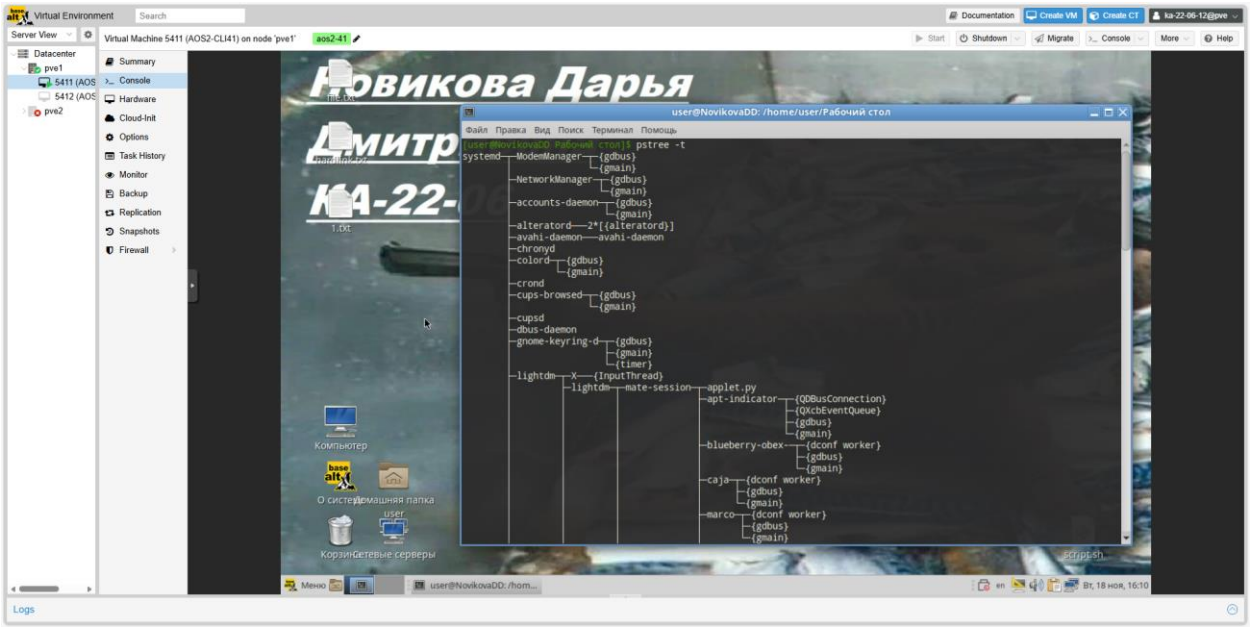

Рисунок 7 — Потоки с именами

Если необходимо скрыть потоки и показать только процессы, используют опцию -T (рис. 8):

$pstree -T

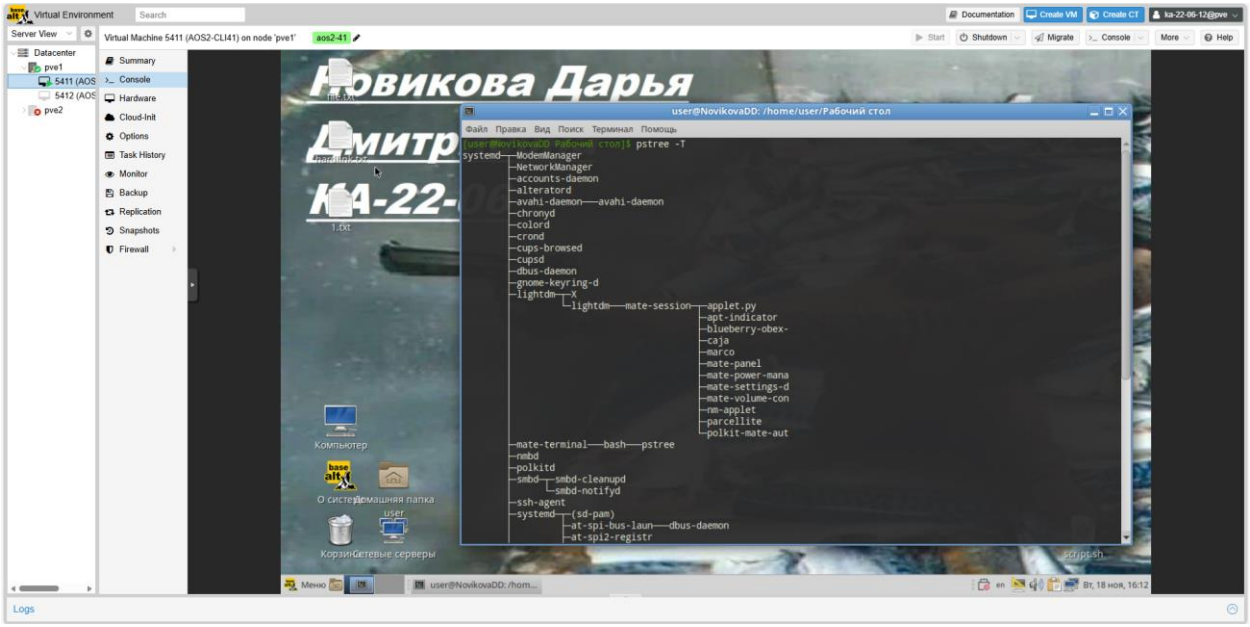

Рисунок 8 — Только процессы

Обычно pstree отображает много строк вывода, которые не помещаются на экране. Чтобы просмотреть вывод по одной странице за раз, передайте его команде less.

Если в качестве аргумента pstree имя пользователя, pstree показывает только процессы, принадлежащие этому пользователю (рис. 9).

$ pstree user

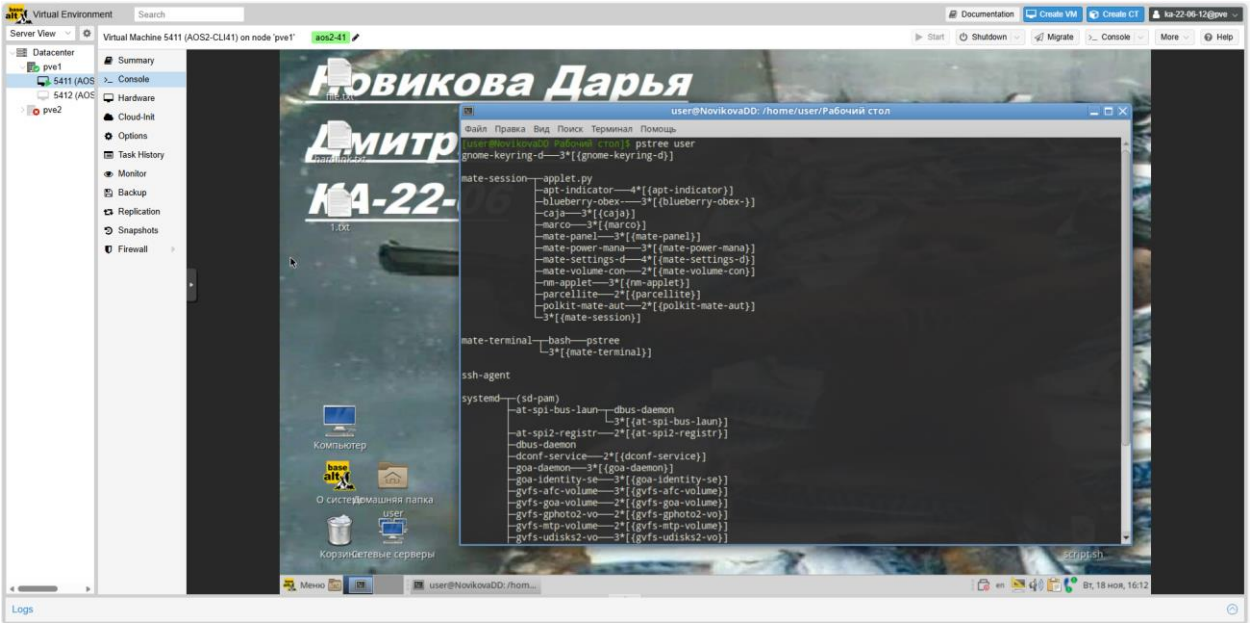

Рисунок 9 — Процессы пользователя

Мониторинг процессов

ps

Манипулируя процессами, система имеет дело именно с их идентификаторами, другого способа отличить один процесс от другого нет. Для просмотра своих процессов можно воспользоваться утилитой ps («process status») (рис. 10):

$ ps –f

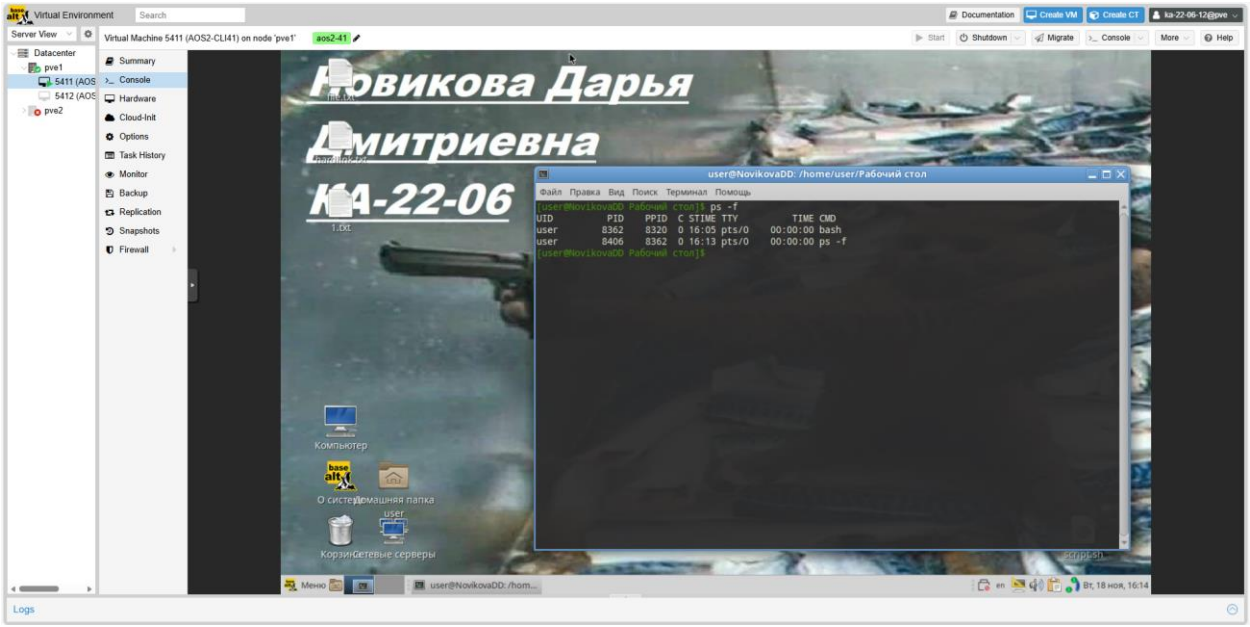

Рисунок 10 — Просмотр процессов

Здесь вызван ps с ключом «-f» («full»), чтобы получить побольше информации.

Команда ps aux показывает все процессы в системе вместе с дополнительной информацией, такой как CPU и память, которую они используют.

Укажите расшифровку каждой характеристике процесса

- PID — уникальный ID процесса
- PPID — ID родительского процесса
- UID/USER — пользователь-владелец
- CMD — команда запуска
- TTY — связанный терминал
- STAT — состояние процесса (R,S,D,T,Z)
- %CPU — загрузка процессора

- %MEM — использование памяти
- VSZ(VIRT) — виртуальная память
- RSS(RES) — физическая память
- STIME — время запуска
- TIME — общее время работы на CPU

top

Интерактивная утилита top отображает список процессов, потребляющих больше всего системных ресурсов (CPU и память), обновляя информацию в реальном времени. Также показывает общую статистику системы. Установлена по умолчанию.

Инструмент top позволяет сортировать список процессов:

Клавиша M – для сортировки по памяти;

Клавиша P – для сортировки по использованию CPU;

Клавиша N – для сортировки по идентификатору процесса;

Клавиша T – для сортировки по времени работы.

По умолчанию top не показывает полный путь к программе и не делает различие между процессами ядра и процессами пользовательского пространства. Если вам нужна эта информация, нажмите **c** (рис. 11)**.** Нажмите c еще раз, чтобы вернуться к отображению по умолчанию.

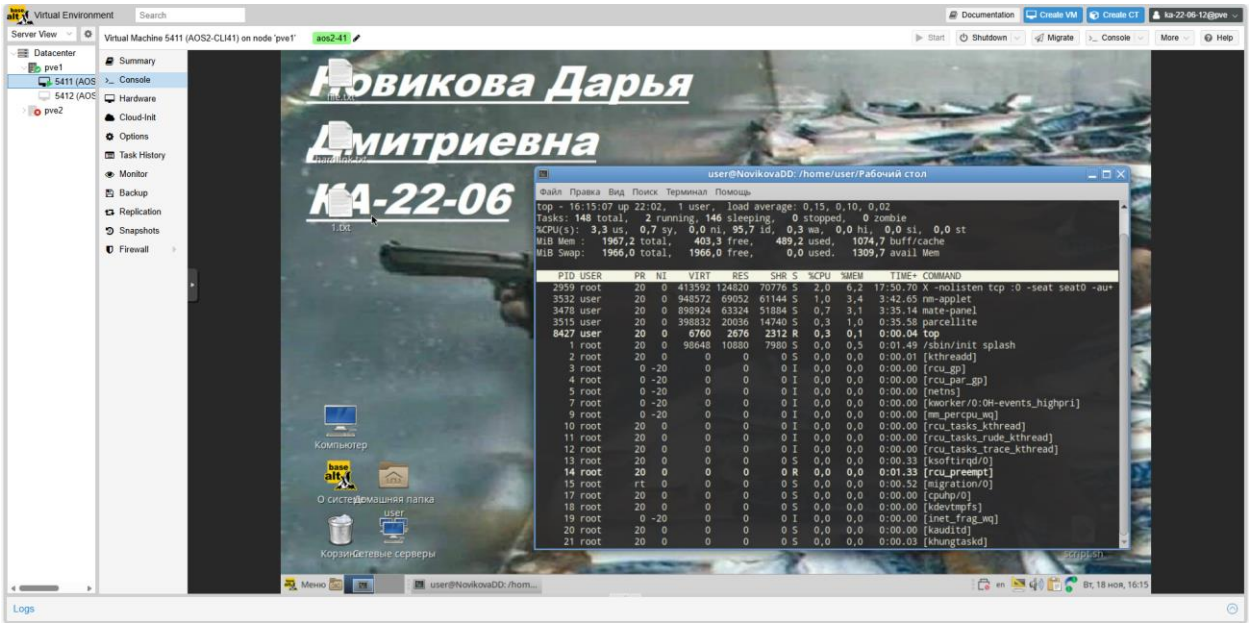

Рисунок 11 — Полный путь к процессу

Иногда в процессе работы может понадобиться иерархия процессов. Вы можете раскрыть ее с помощью клавиш **V** во время работы top (рис. 12).

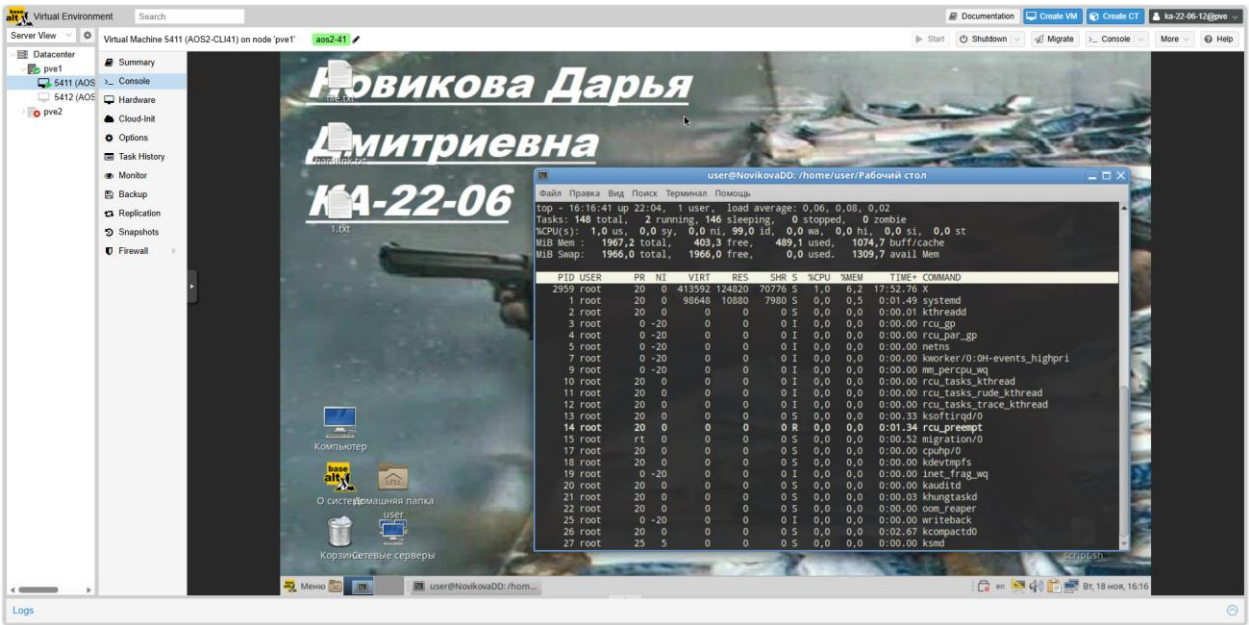

Рисунок 12 — Иерархия процессов

Чтобы вывести процессы определенного пользователя, нажмите **u**, затем введите имя пользователя или оставьте его пустым, чтобы отобразить процессы для всех пользователей (рис. 13).

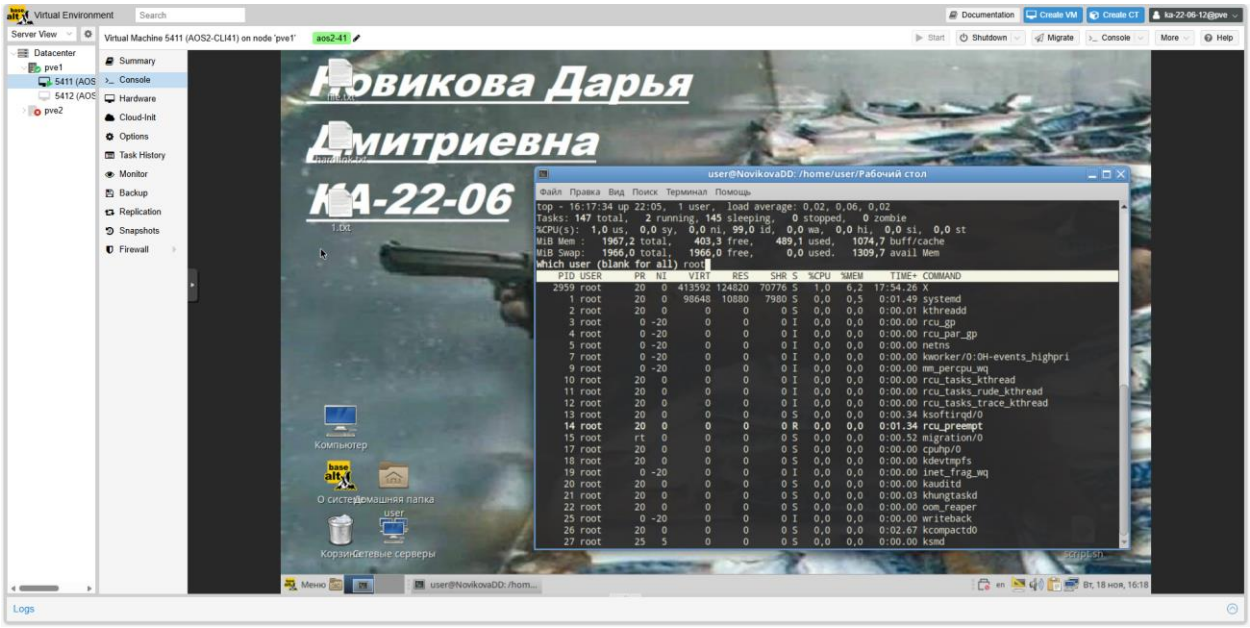

Рисунок 13 — Процессы по пользователю

Вы можете нажать **t и m**, чтобы изменить стиль статистики процессора и памяти (рис. 14).

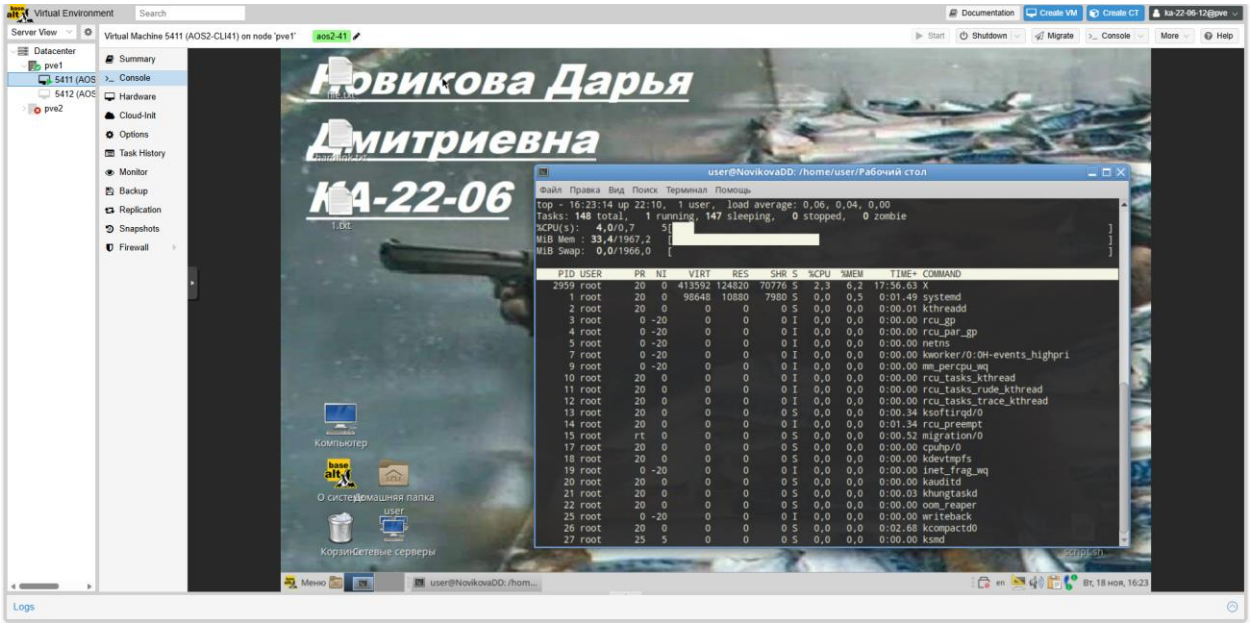

Рисунок 14 — Стиль отображения

Интерфейс можно разделить на две рабочие зоны:

Верхняя рабочая зона содержит сведения о времени работы сервера, свободных и занятых ресурсах, пользователях.

Основная рабочая зона – динамически обновляемая таблица, содержащая сведения о процессах.

На первой строке показаны следующие поля:

- текущее время;
- время безотказной работы системы;
- количество активных сеансов пользователей;
- средняя загрузка системы (load average) – три величины, усредненные за последние 1, 5 и 15 минут. Чем ниже, тем лучше. Если превышает количество ядер на сервере, то значит сервер перегружен.

В разделе Tasks отображается статистика процессов, выполняемых в вашей системе. Далее перечислено общее количество процессов, активные, спящие, остановленные и процессы-зомби.

Раздел использования CPU показывает процентное время процессорного времени, затрачиваемого на различные задачи:

- us – потребление CPU пользовательскими процессами (top);
- sy – потребление CPU системными процессами;
- ni – потребление CPU пользовательскими процессами, с измененным приоритетом (командой nice или renice);
- id – простаивание CPU;
- wa – процент времени, в течение которого процессор простаивал, ожидая завершения операции ввода-вывода. Если слишком высокое (более 10), значит за вашим процессором не поспевает диск, и нужно бы поменять его на более быстрый;
- hi – процент времени работы аппаратных прерываний. Периферийные устройства могут прерывать работу процессора, чтобы передать какие-то данные;
- si – процент времени работы программных прерываний. Некоторые приложения могут прерывать работу процессора, чтобы передать какие-то данные;
- st – процент времени, когда реальный процессор не был доступен виртуальной машине. Будет расти если виртуальной машине выделить больше ядер, чем имеет гипервизор. Этот показатель имеет значение только в системах виртуализации.

Проведите анализ нагрузки на вашей системе. Интерпретируйте каждый параметр (рис. 15).

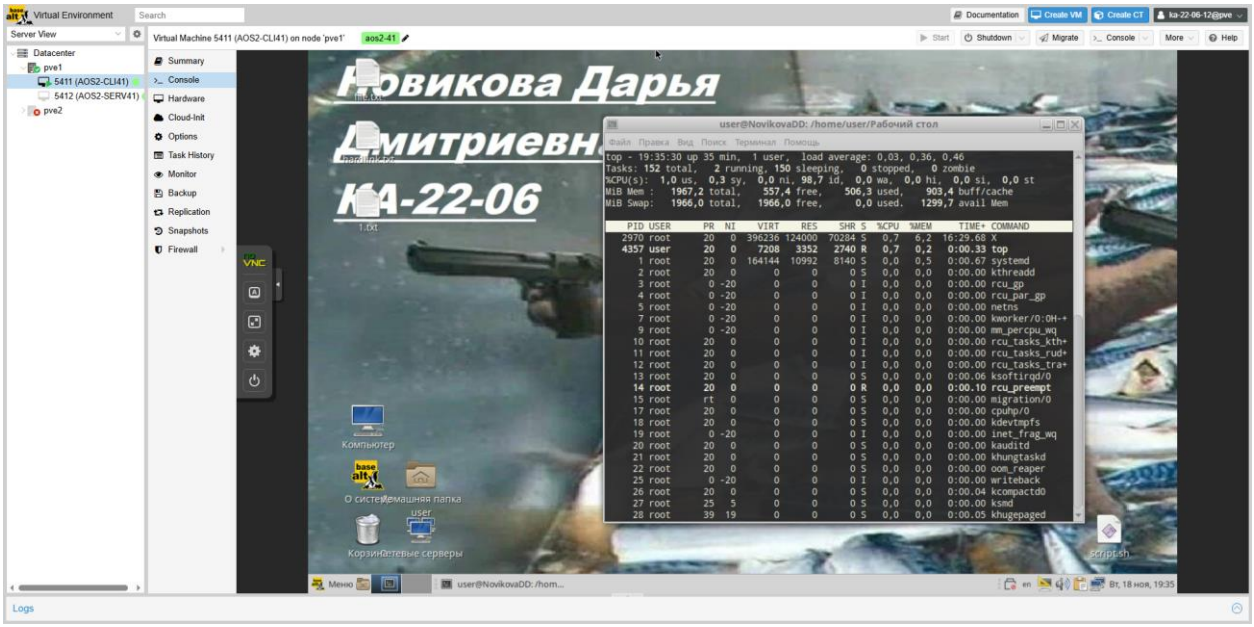

Рисунок 15 — Анализ нагрузки системы

Процессы: всего — 152, из них выполняются — 2, остальные — в состоянии ожидания. Зомби- и остановленных процессов нет.

Загрузка CPU:

us = 18,9% — пользовательские процессы,

sy = 2,4% — системные вызовы,

id = 78,8% — простой CPU,

wa = 0,0% — отсутствие ожидания ввода-вывода,

ni = hi = si = st = 0,0% — нет процессов с изменённым приоритетом, прерываний или эффекта виртуализации.

Память: используется 506 МБ из 1967 МБ (≈25%), своп не задействован.

Вывод: система работает в штатном режиме с низкой нагрузкой; ресурсов (CPU, памяти, диска) достаточно.

Последние 2 строки показывают информацию об использовании памяти в системе. Строки Mem и Swap отображают информацию о RAM и области подкачки соответственно.

total – общее количество памяти;

free – количество свободной памяти;

used – количество используемой памяти;

buff / cache – память, выделенная под буфер и дисковый кэш;

avail Mem – память, которая может быть выделена для запуска новых процессов.

Htop

Утилита htop является улучшенной версией top, предоставляющей более удобный пользовательский интерфейс, возможности для взаимодействия с процессами (например, завершение процессов) и более гибкие настройки отображения.

Управление:

F1 – справка;

F2 – настройки;

F3 – поиск процесса;

F4 – сортировка списка процессов;

F5 – устанавливает древовидное отображение;

F6 – открывает панель с выбором параметра сортировки процессов;

F7 – увеличивает приоритет выполнения текущего процесса;

F8 – уменьшает приоритет выполнения текущего процесса;

F9 – передать сигнал процессу;

F10 – выходит из программы.

Кнопка Home перенесёт к началу списка.

Кнопка End прокрутит список до конца.

Ctrl-a или ^ – прокрутка к началу пункта (началу строки).

Ctrl-e или $ – прокрутка к концу пункта записи о процессе (то есть концу строки).

Приоритеты выполнения процессов

Niceness (уступчивость)

Представляет собой значение приоритета, которое процессы могут устанавливать, чтобы уступать CPU другим процессам. Значение niceness варьируется от -20 (высокий приоритет) до 19 (низкий приоритет). Утилиты

top и htop позволяют изменять значение niceness для процессов в реальном времени, что может быть полезно для управления нагрузкой на систему.

NICE может быть в пределах от -20 до 19. По умолчанию 0.

Утилита nice используется для задания приоритета процесса.

Пример (в терминале) (рис. 16):

$ nice -n 15 ping ya.ru > /dev/null &

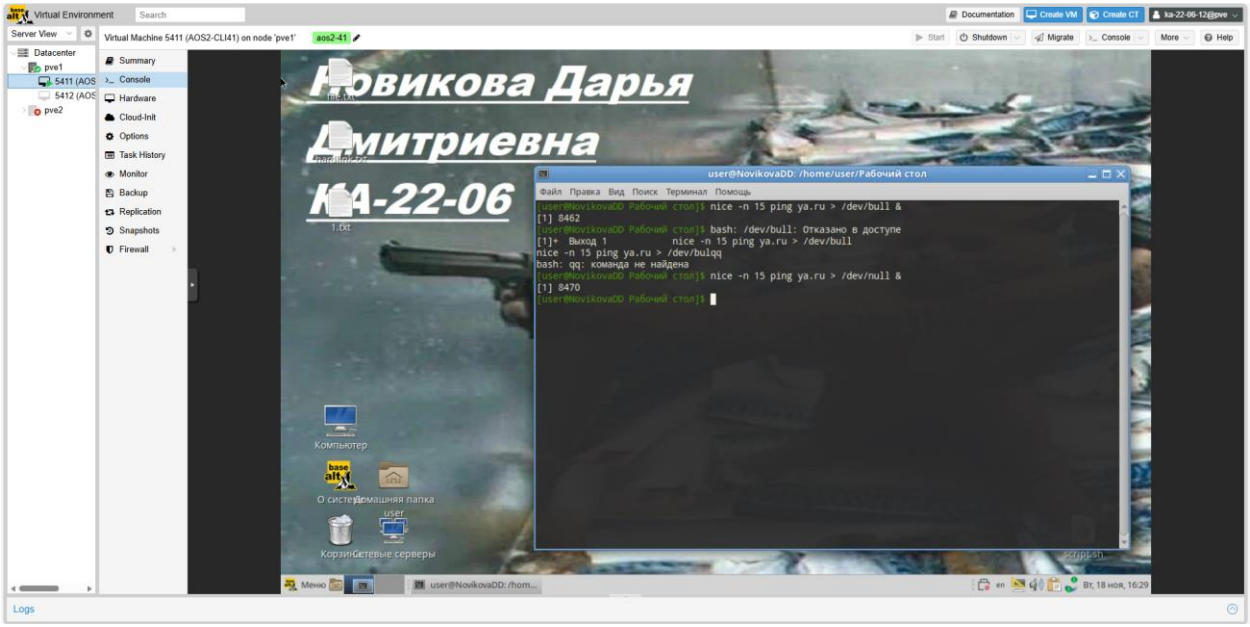

Рисунок 16 — Процесс с низким приоритетом

$ ps ax -o pid,ppid,cmd,nice,etime

Результат выполнения (рис. 17):

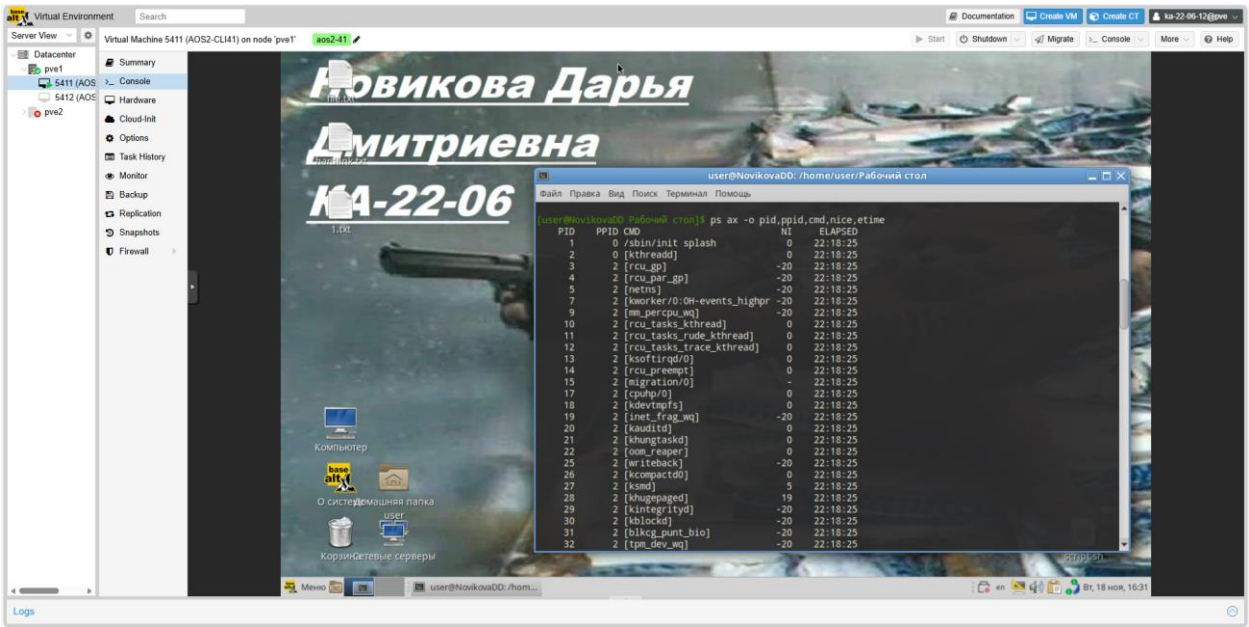

Рисунок 17 — Просмотр приоритетов

Утилита renice используется для изменения приоритета процесса.

Пример (в терминале) (рис. 18):

$ renice 10 3833

$ renice 10 $(pgrep firefox)

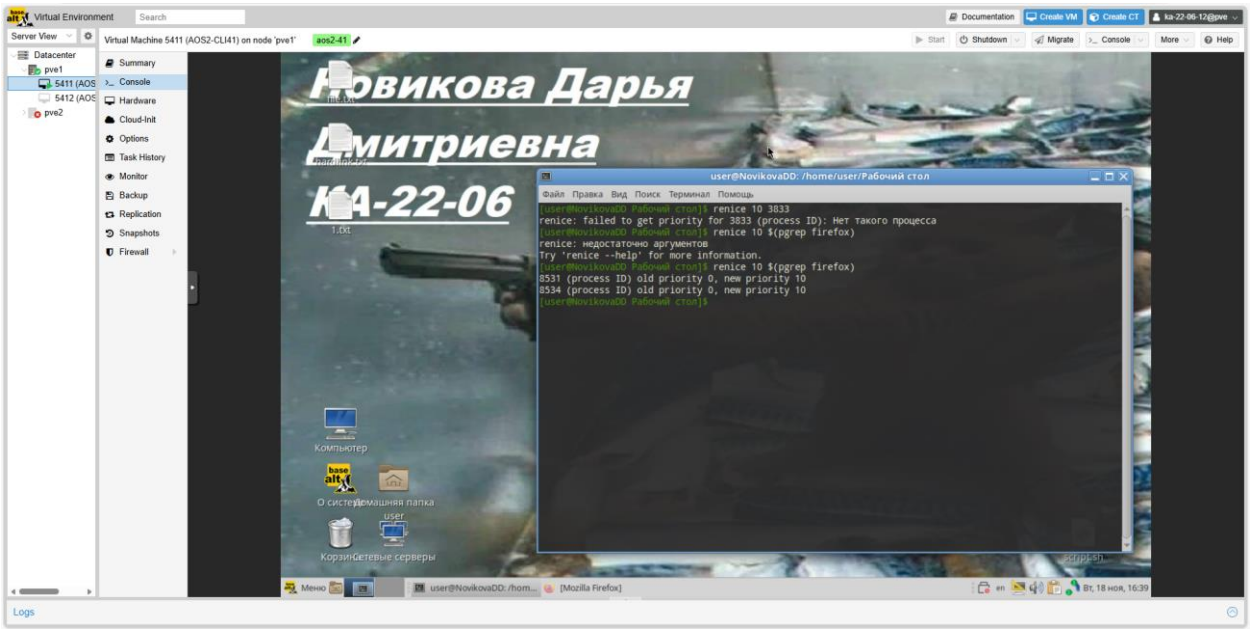

Рисунок 18 — Смена приоритета

Сигналы

В Linux сигналы используются для управления процессами. Сигналы – это асинхронные уведомления, которые могут отправляться одному процессу (процессу-получателю) от другого процесса (процесса-отправителя) или от операционной системы.

Сигналы могут быть:

- Обрабатываемые самим процессом (игнорируемые)

Процессы могут быть запрограммированы на обработку определённых сигналов специфическим образом, включая их игнорирование.

Некоторые сигналы, такие как SIGKILL и SIGSTOP, не могут быть перехвачены или обработаны процессами и всегда обрабатываются ядром.

- Обрабатываемые ядром

Процессы могут установить обработчики сигналов, чтобы определить поведение при получении конкретного сигнала, например, процесс может обработать сигнал SIGTERM для корректного завершения работы. Кроме того,

сигналы могут использоваться для синхронизации процессов, уведомления о событиях или для обеспечения безопасного завершения процессов.

Чтобы завершить работу фонового процесса с помощью ≪^C≫, необходимо сначала сделать его активным. Это не всегда возможно, и не всегда удобно. Сигнал – это способность процессов обмениваться стандартными короткими сообщениями непосредственно с помощью системы. Сообщение-сигнал не содержит никакой информации, кроме номера сигнала (для удобства вместо номера можно использовать предопределенное системой имя). Для того, чтобы передать сигнал, процессу достаточно задействовать системный вызов **kill(),** а для того, чтобы принять сигнал, не нужно ничего. Если процессу нужно как-то по-особенному реагировать на сигнал, он может зарегистрировать обработчик, а если обработчика нет, за него отреагирует система. Как правило, это приводит к немедленному завершению процесса, получившего сигнал. Обработчик сигнала запускается асинхронно, немедленно после получения сигнала, чтобы процесс в это время ни делал. сигнал Короткое сообщение, посылаемое системой или процессом другому процессу. Обрабатывается асинхронно специальной подпрограммой-обработчиком. Если процесс не обрабатывает сигнал самостоятельно, это делает система.

Два сигнала – 9 (KILL) и 19 (STOP) – всегда обрабатывает система. Первый из них нужен для того, чтобы убить процесс наверняка (отсюда и название). Сигнал STOP приостанавливает процесс: в таком состоянии процесс не удаляется из таблицы процессов, но и не выполняется до тех пор, пока не получит сигнал 18 (CONT) – после чего продолжит работу. В Linux сигнал STOP можно передать активному процессу с помощью управляющего символа ≪^Z≫:

Посылка сигналов:

$ kill [-SIGNAL] PID

Приведите 2 примера “убийства” процесса браузере Firefox в системе по номеру сигнала и названию. Использовать сигнал SIGKILL. Листинг команд приведите ниже.

1) Сначала найдем PID (идентификатор) процесса Firefox. Можно использовать команду pgrep (рис. 19).

kill -9 $(pgrep firefox)

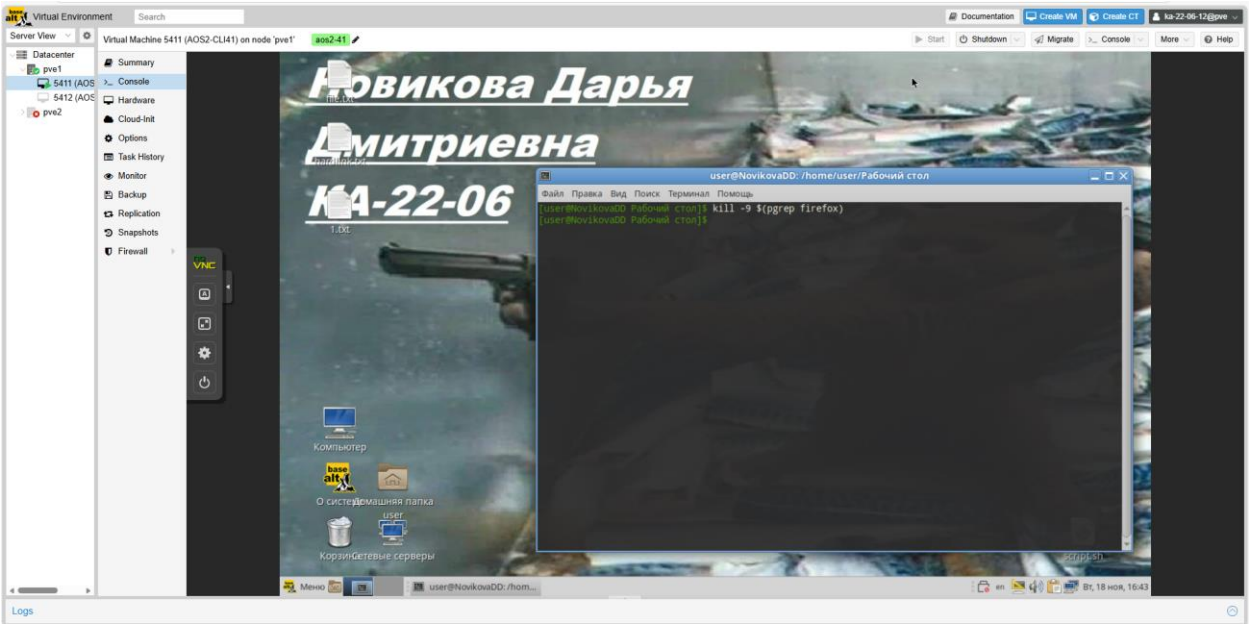

Рисунок 19 — Жесткое завершение

2) kill -SIGKILL $(pgrep firefox)

Результат (рис. 20):

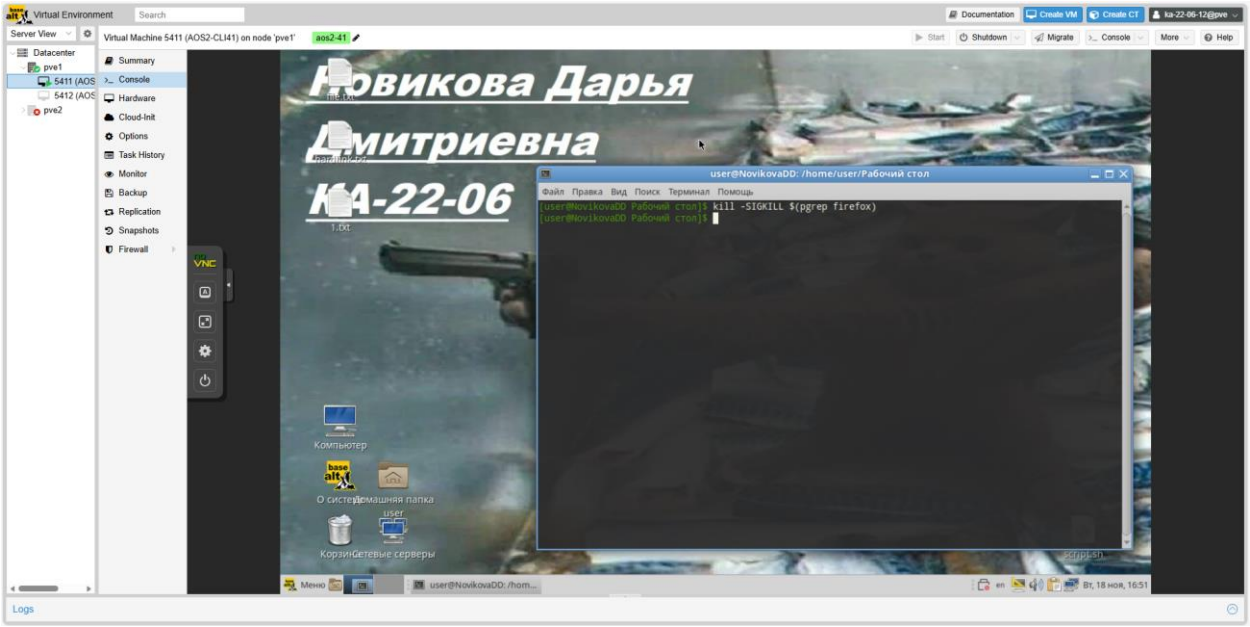

Рисунок 20 — Сигнал принудительного завершения

$ killall [-SIGNAL] { user | name }

Запустите множество утилит на ваш выбор. Сначала “остановите” все процессы текущего пользователя, а затем их возобновите. Листинг команд приведите ниже.

killall -STOP -u $USER

killall -CONT -u $USER

$ pkill [options] pattern

Запустите браузер Firefox и почтовый сервис ThunderBird. “Убейте” их. Листинг команд приведите ниже (рис. 21).

pkill firefox

pkill thunderbird

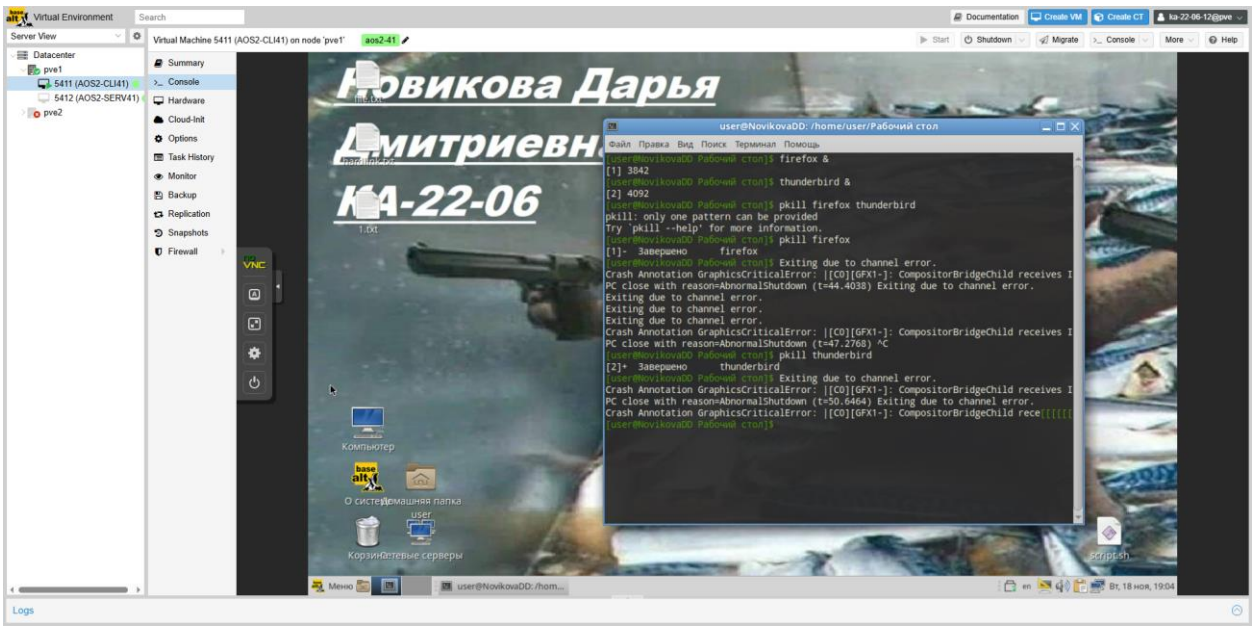

Рисунок 21 — Завершение по имени

Часто используемые сигналы:

№	Имя	Описание
1	HUP	обрыв связи,
2	INT	Прервать (Ctrl+C)
9	KILL	Уничтожить
15	TERM	Завершить (сигнал по умолчанию)
18	CONT	Продолжить после STOP
19	STOP	Приостановить
20	TSTP	Стоп с клавиатуры (Ctrl+Z)

КОНТРОЛЬНЫЕ ЗАДАНИЯ

1. Используя команду ps, отобразите все процессы, запущенные текущим пользователем и отдельно пользователем root. Найдите процессы, которые работают в фоновом режиме.

Результат выполнения задания представлен на рисунках 22, 23 и 24.

$ ps -u $user

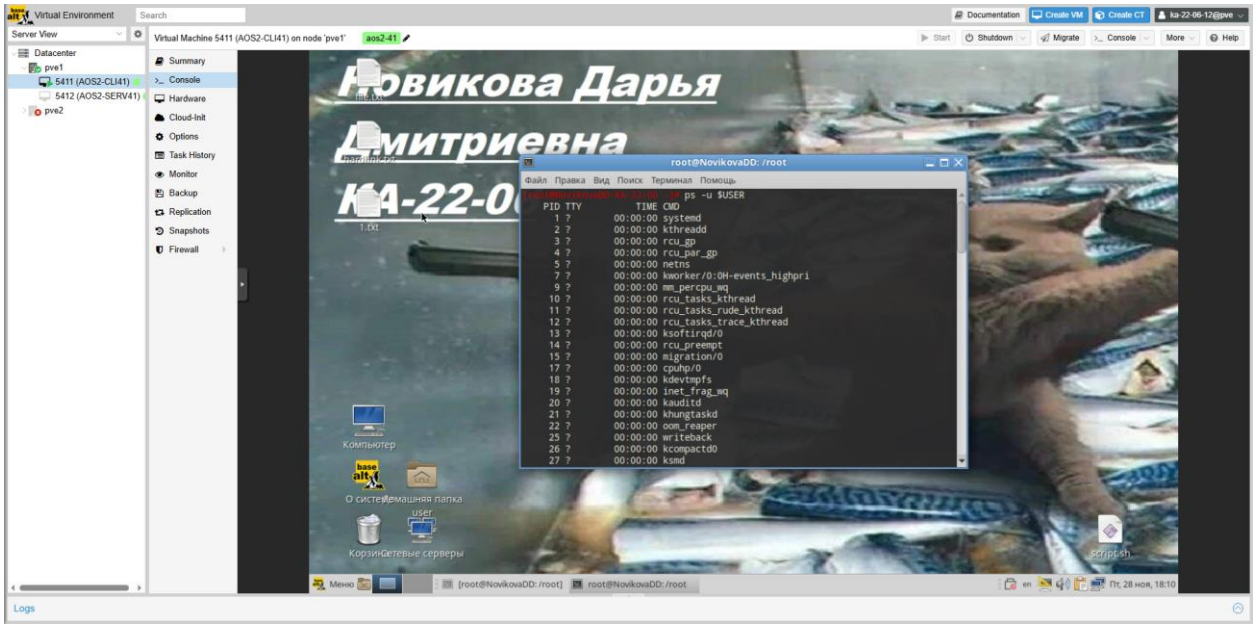

Рисунок 22 — Задание 1

$ ps -u $root

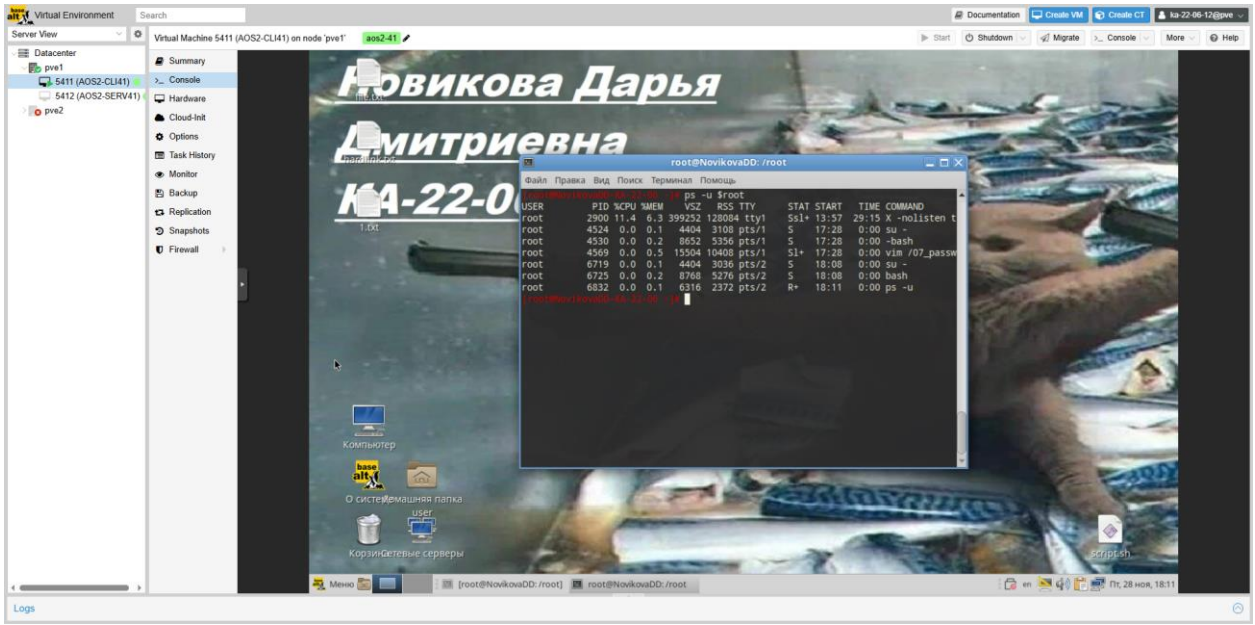

Рисунок 23 — Задание 1

$ps -e | grep pts

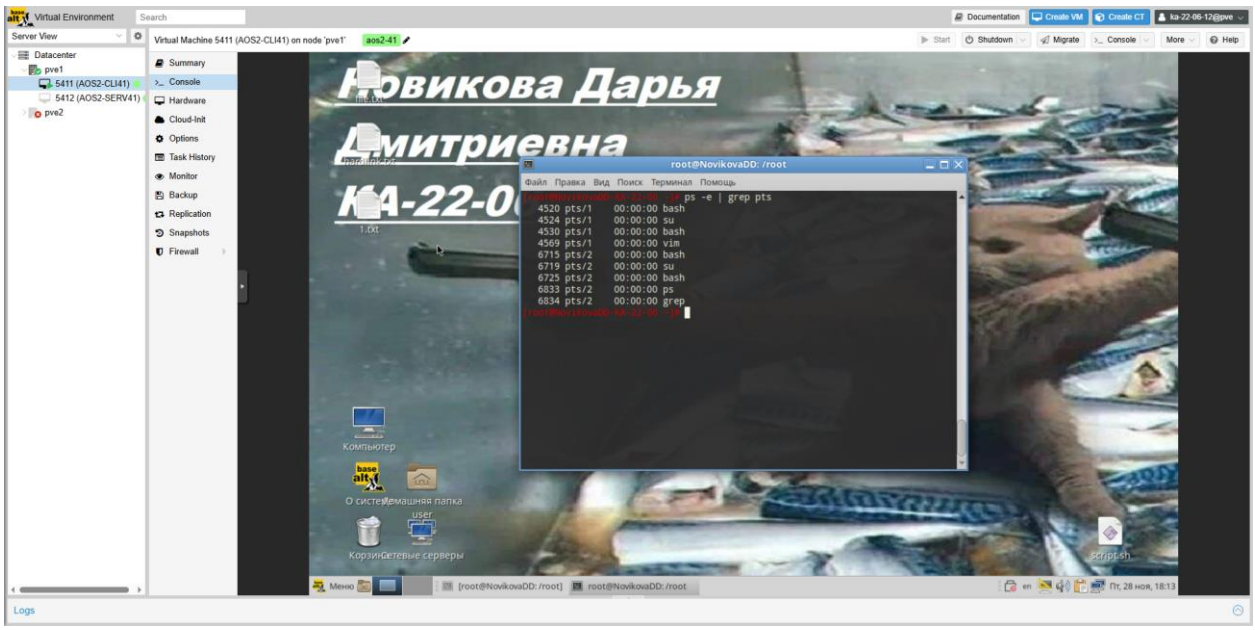

Рисунок 24 — Задание 1

2. С помощью команды top или htop отследите потребление ресурсов текущими процессами (CPU, память) и выведите топ-5 процессов, потребляющих наибольшее количество CPU.

Открыть утилиту *top*

Нажать клавишу *P* (заглавная латинская). Процессы сразу же пересортируются, и вверху окажутся те, что используют больше всего процессорного времени.

Результат выполнения задания представлен на рисунке 25.

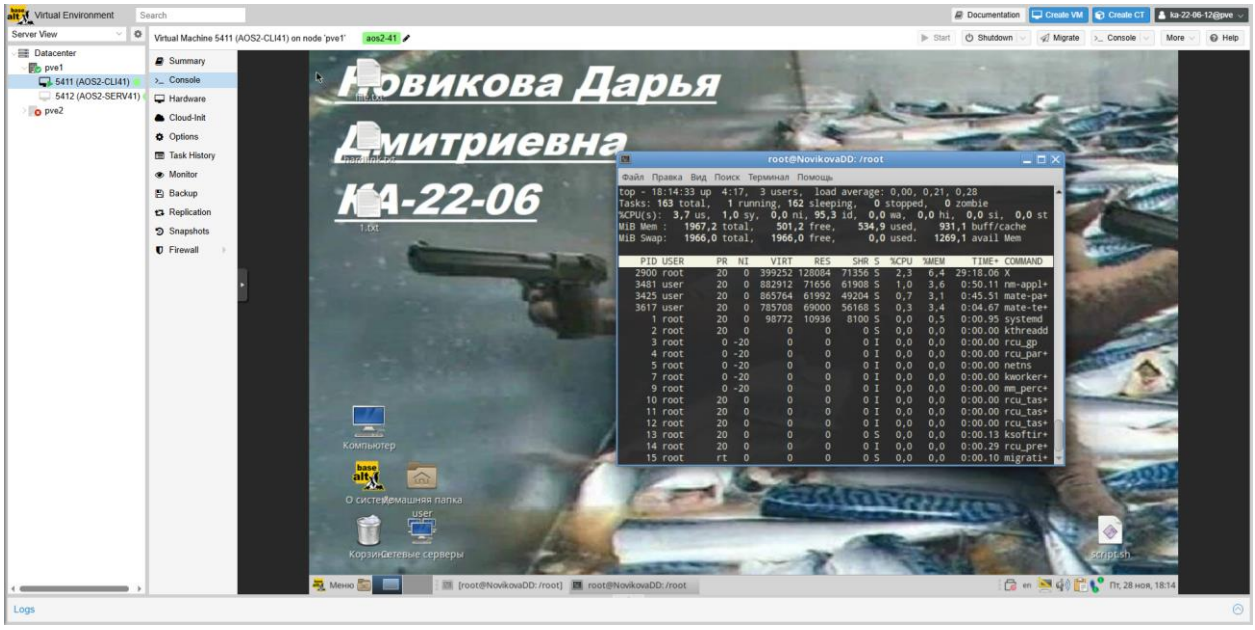

Рисунок 25 — Задание 2

3. Запустите долгий процесс (создайте для этого скрипт, который будет бесконечно выполняться) и переведите его в фоновый режим. Остановите выполнение процесса, а затем возобновите его выполнение с помощью “команд управления заданиями”.

cd ~

nano infinite.sh

Делаем исполняемым: *chmod +x infinite.sh*

Запускаем в фоне *./infinite.sh &*

Видим, процессу присвоился идентификатор [6897]

Остановить *kill -STOP 6897*

Возобновить *kill -CONT 6987*

Результат выполнения задания представлен на рисунках 26 и 27.

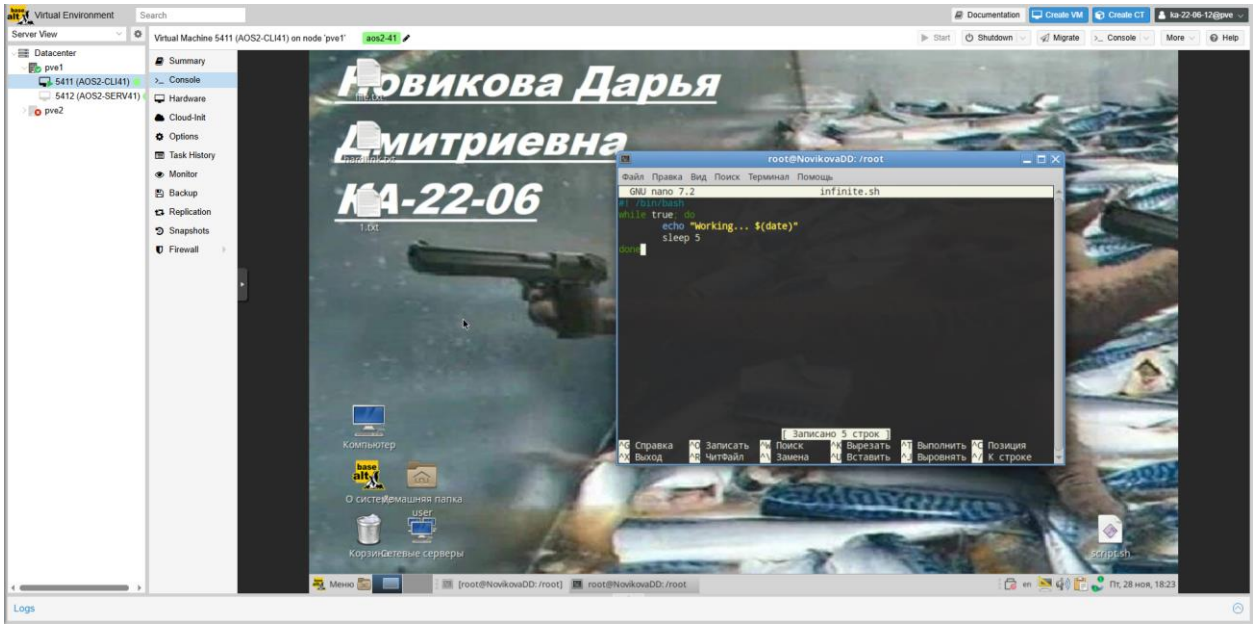

Рисунок 26 — Задание 3

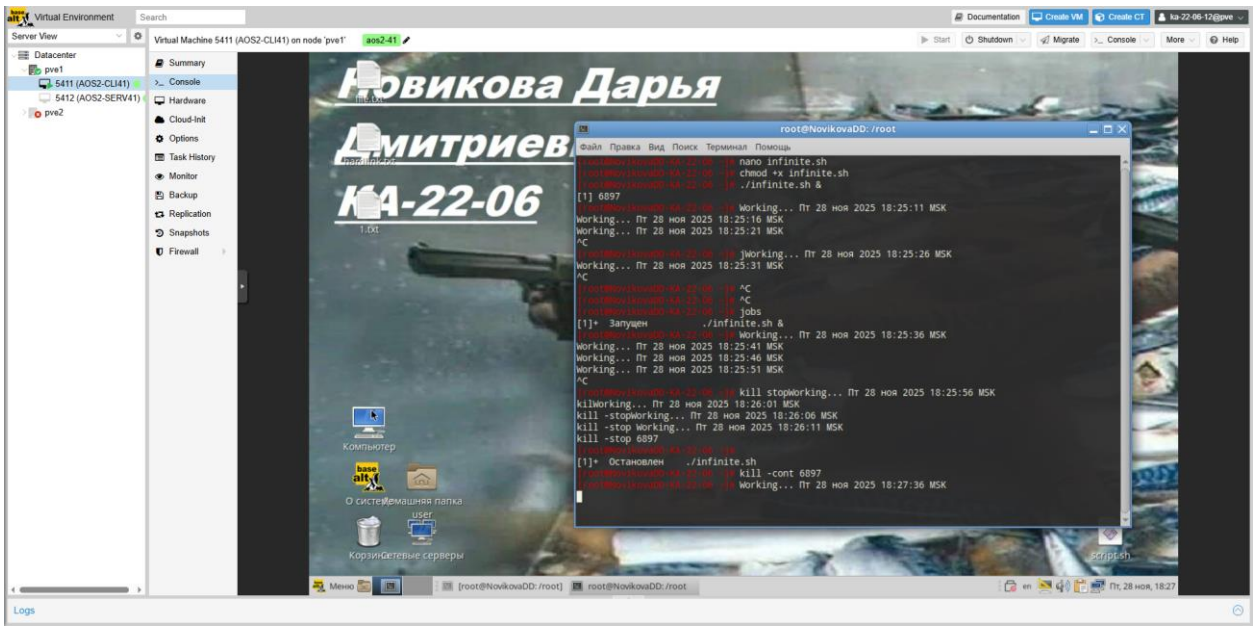

Рисунок 27 — Задание 3

4. Измените приоритет процесса так, чтобы он был стал самым значимым в системе. Проанализируйте количество выделяемых на него ресурсов до изменения и после.

Запускаем скрипт в фоновом режиме с максимально высоким приоритетом в системе

nice -n -20 ./infinite.sh &

Результат выполнения задания представлен на рисунке 28.

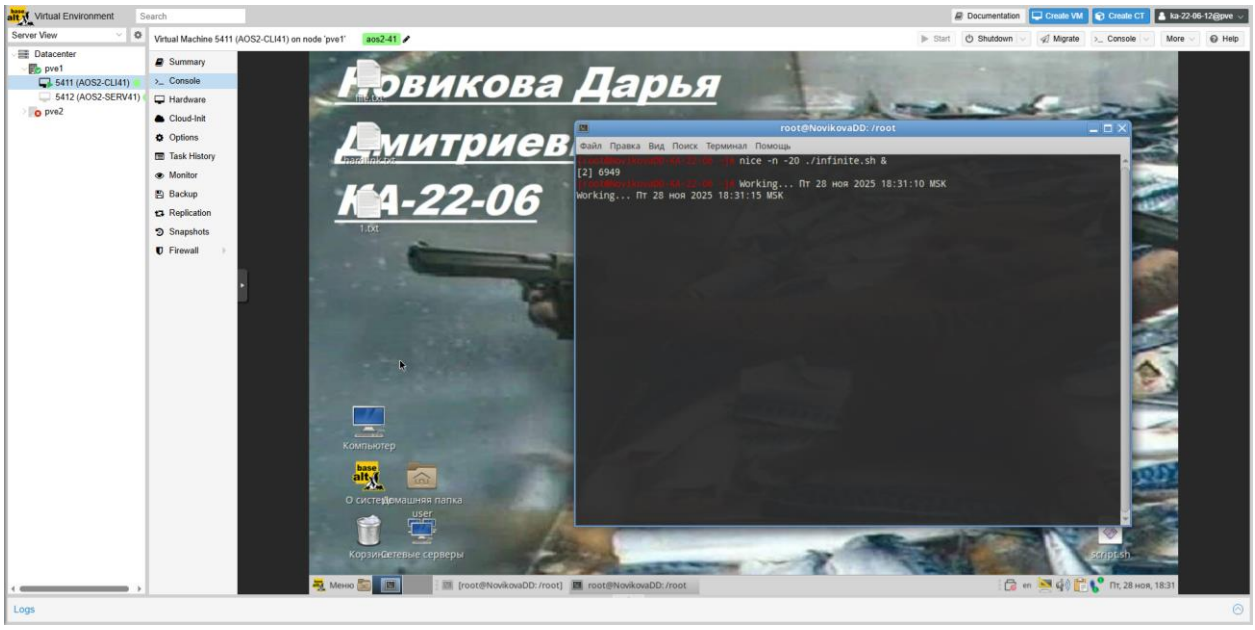

Рисунок 28 — Задание 4

5. С помощью утилиты htop измените приоритет выполнения любого процесса на самый минимальный и максимальный.

В терминале *htop*

Выберите процесс клавишами ↑/↓

F7 (увеличить приоритет) или F8 (уменьшить приоритет)

Или F9 и выберите сигнал для отправки

Результат выполнения задания представлен на рисунке 29.

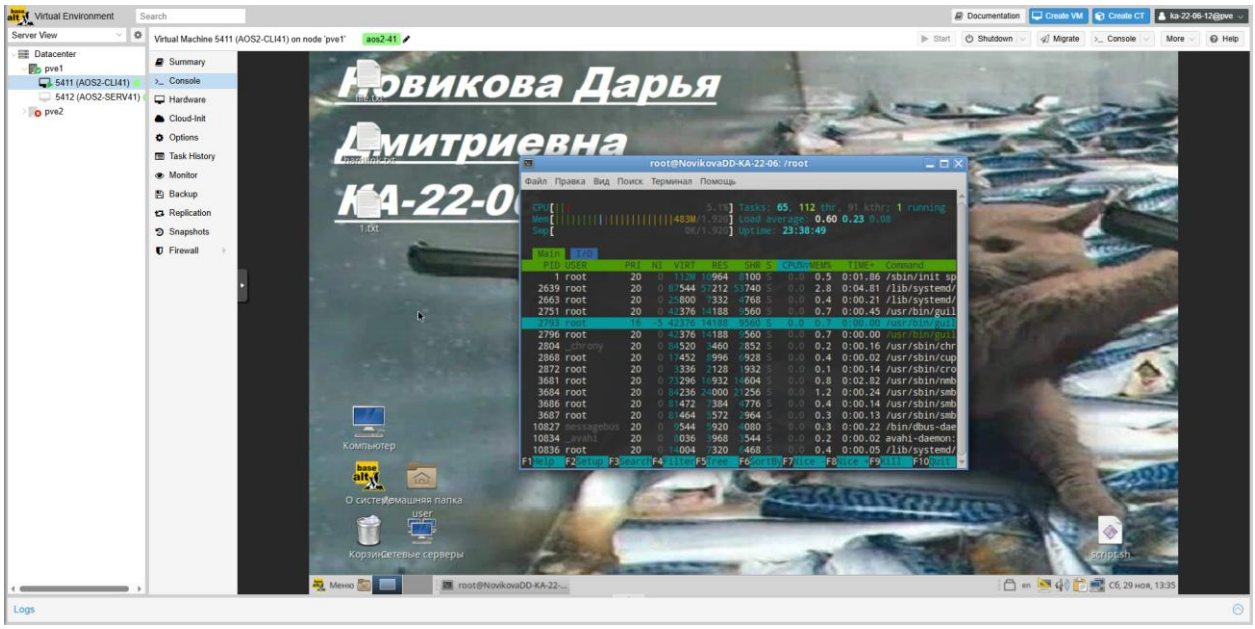

Рисунок 29 — Задание 5

6. Сконфигурируйте меню htop, добавив колонки, которые будут показывать ID родителя процесса и терминал, на котором выполняется процесс.

F2 (Setup)

Выберите "Columns"

Добавьте колонки:

PPID (Parent Process ID)

TTY (Terminal)

F10 для сохранения

Результат выполнения задания представлен на рисунке 30.

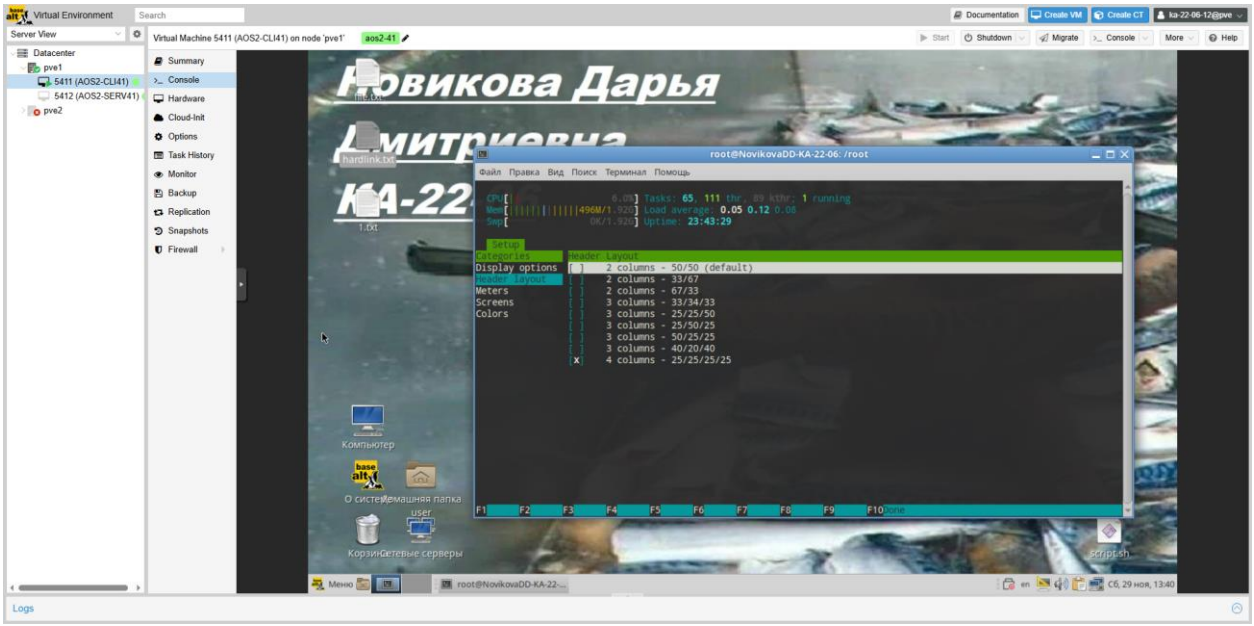

Рисунок 30 — Задание 6

7. В htop проанализируйте с помощью “поиска” или “фильтрации”, какие процессы запускаются при открытии браузера Firefox. Что произойдет, если начать писать или выполнять поиск в браузере? На какие процессы и библиотеки системы выделяет большее количество ресурсов?

F3 (Поиск)

Введите "firefox"

Запустите Firefox и наблюдайте за новыми процессами

Результат выполнения задания представлен на рисунке 31.

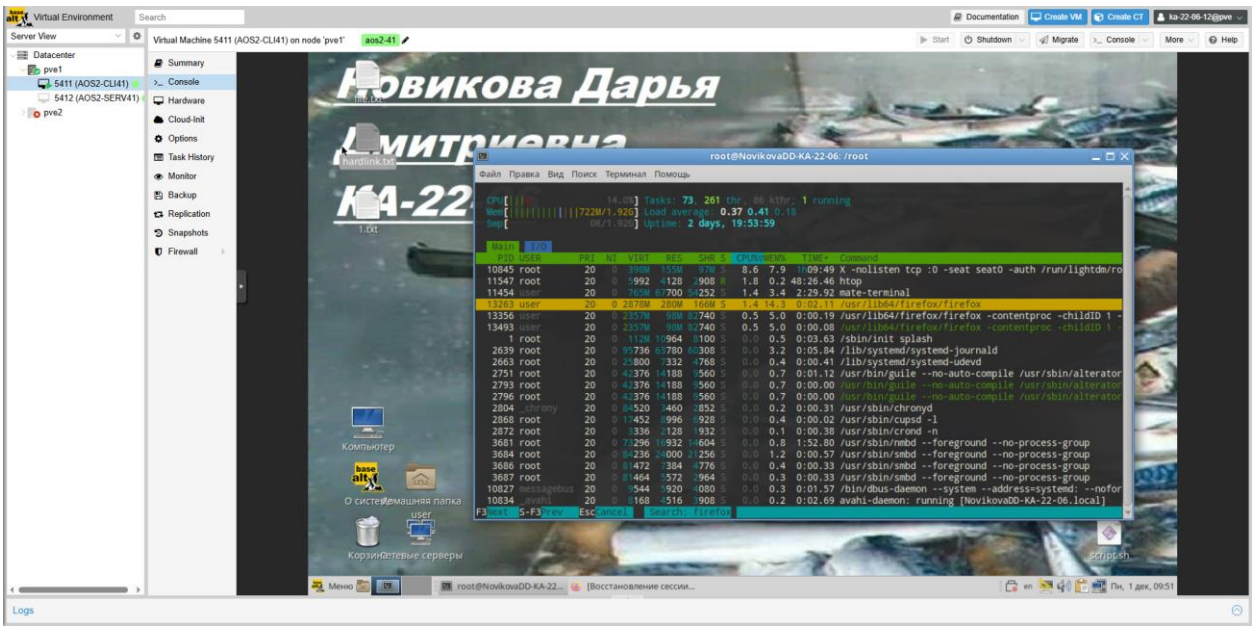

Рисунок 31 — Задание 7

8. С помощью утилиты htop отправьте сигнал остановки запущенному “бесконечному” скрипту.

Найти скрипт в меню

F9 (Kill) — откроется меню отправки сигналов

В левой панели выбрать сигнал:

- SIGSTOP (19) — временная остановка процесса
- SIGTSTP (20) — остановка с терминала

Нажмите Enter для отправки сигнала

Результат выполнения задания представлен на рисунке 32.

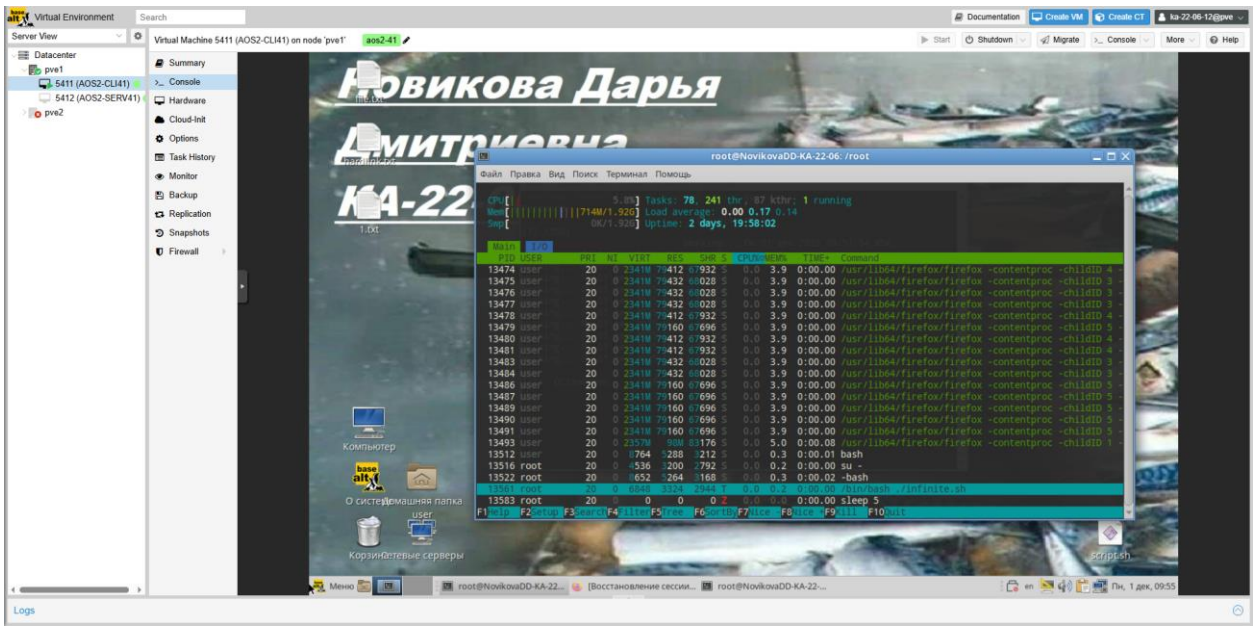

Рисунок 32 — Задание 8

9. Найдите в вашей системе процессы-зомби. Почему вы смогли/не смогли их найти?

В *htop*

F5 для древовидного отображения

Процессы со статусом Z или Z+

Через *top* можно посмотреть в строке статистики: zombie (показывается количество)

Результат выполнения задания представлен на рисунках 33, 34.

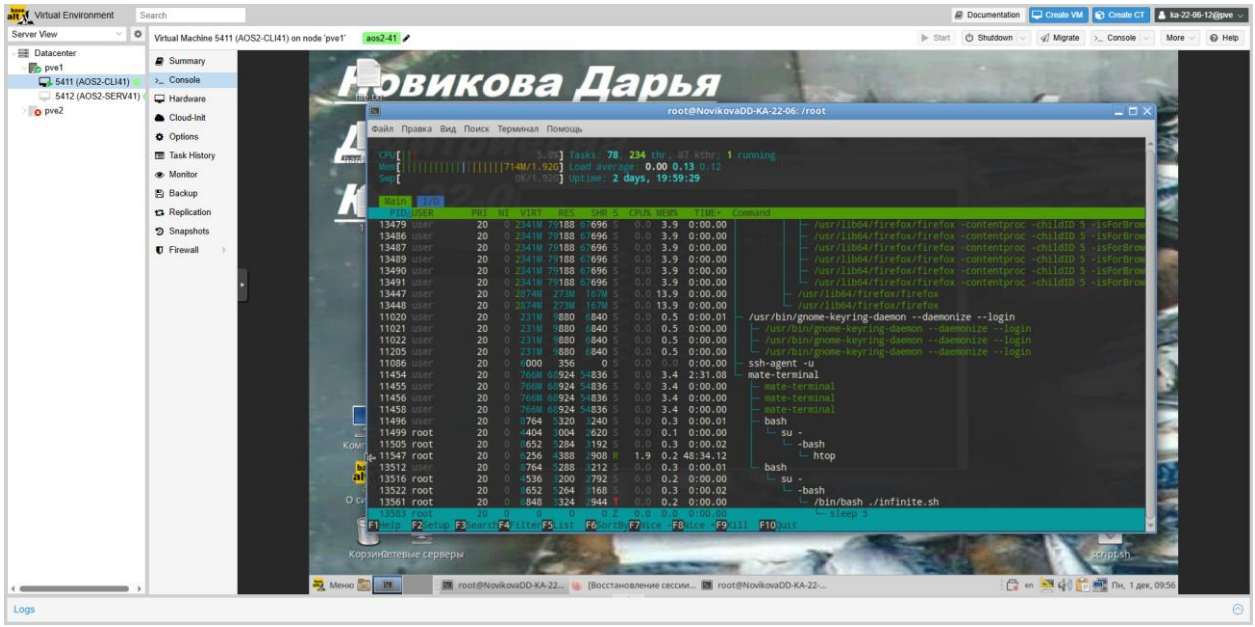

Рисунок 33 — Задание 9

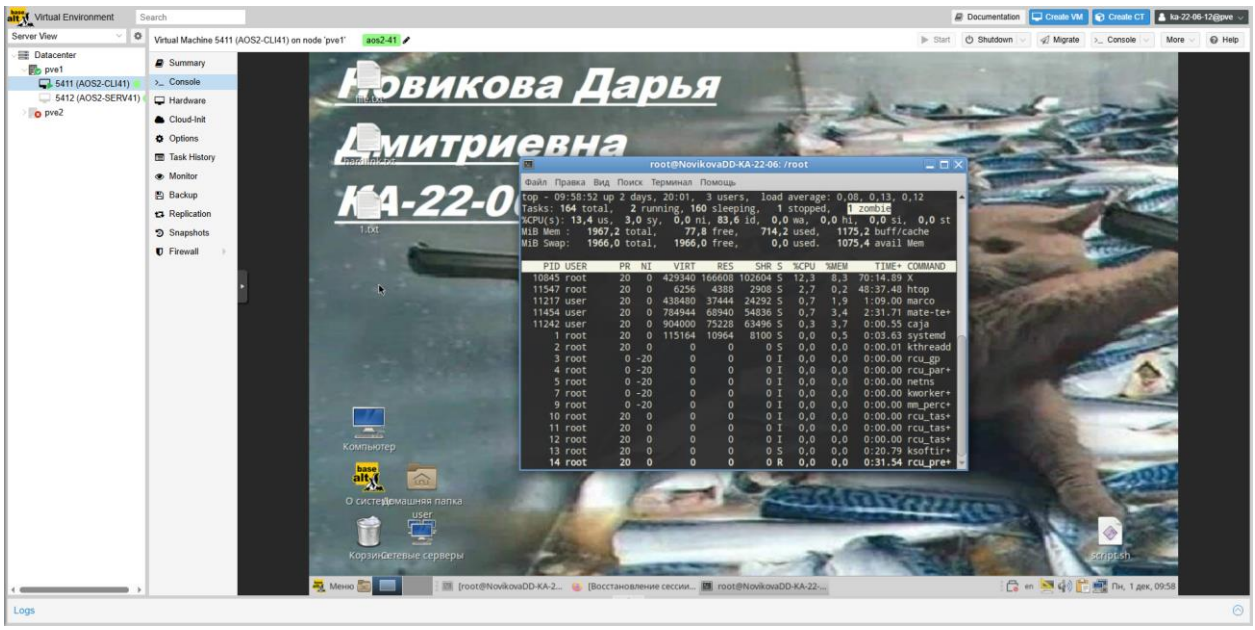

Рисунок 34 — Задание 9

Причины отсутствия зомби:

- Ядро и программы правильно обрабатывают завершение дочерних процессов
- Зомби процессы обычно существуют очень короткое время
- Нет проблемных программ
- На системе мало запущенных/завершающихся процессов

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены основные принципы управления процессами. Рассмотрены ключевые аспекты работы с процессами, включая их мониторинг, управление приоритетами и передачу сигналов. Полученные навыки позволяют эффективно управлять системными ресурсами, контролировать выполнение процессов и оперативно реагировать на различные ситуации в работе операционной системы.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое процесс в операционной системе? Какие основные состояния процесса существуют?

Процесс — это экземпляр выполняющейся программы. Основные состояния:

- Running (выполняется) — процесс использует CPU
- Sleeping (спящий) — ожидает ресурсов/событий
- Stopped (остановлен) — приостановлен сигналом (Ctrl+Z)
- Zombie (зомби) — завершен, но не удален из таблицы процессов

2. Что такое приоритет процесса? Как изменить приоритет уже запущенного процесса?

Приоритет (nice) определяет очередность доступа к CPU. Диапазон: -20 (высший) до 19 (низший).

Изменить приоритет запущенного процесса:

renice -n 5 PID — Установить значение 5

renice -n -5 $(pgrep firefox) — Для процесса Firefox

3. Что такое zombie-процессы, и как они возникают? Как их устранить?

Zombie-процессы — завершенные процессы, оставшиеся в таблице процессов из-за того, что родительский процесс не прочитал их статус завершения.

Устранение:

1. Перезапустить родительский процесс
2. Отправить SIGCHLD родителю: kill -CHLD PPID
3. Принудительно завершить родителя, если zombie остаются

4. Что такое фоновые и приостановленные процессы? Как с ними взаимодействовать с использованием команд оболочки?

Фоновые — процессы, выполняющиеся без контроля терминала.

Приостановленные — процессы, временно остановленные сигналом STOP.

Команды управления:

команда &	Запуск в фоне
Ctrl + Z	Приостановить процесс
jobs	Список задач
fg %1	Вернуть на передний план
bg %1	Запустить в фоне
kill %1	Завершить задачу

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Официальная документация операционной системы Альт Рабочая станция 10.4 [Электронный ресурс] – URL: https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html/alt-workstation/index.html